

# Scorm: AST Generation

Teacher: Dr. Nguyen Hua Phung

## Introduction

- Cây ít node, nhỏ hơn, gọn nhẹ hơn so với parse tree
- Vì vậy, ta có thể thao tác nhanh hơn ở các bước tiếp theo.

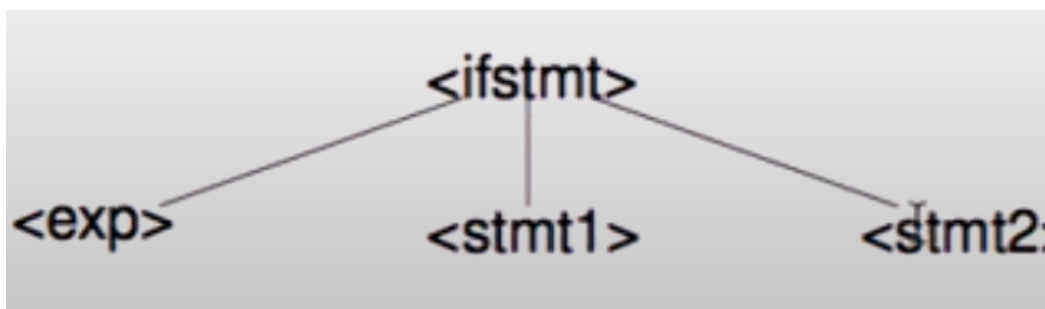
### Ex1

- Ví dụ, với cây parse tree sau:



Parse tree

- Cây này có 1 node root <if\_stmt> và node này có tới 6 node con.
- AST Tree Sẽ rút gọn parse tree này lại chỉ còn 1 root và 3 node con, loại bỏ đi các token như IF, THEN, ELSE để tạo ra cây mới:



AST Tree

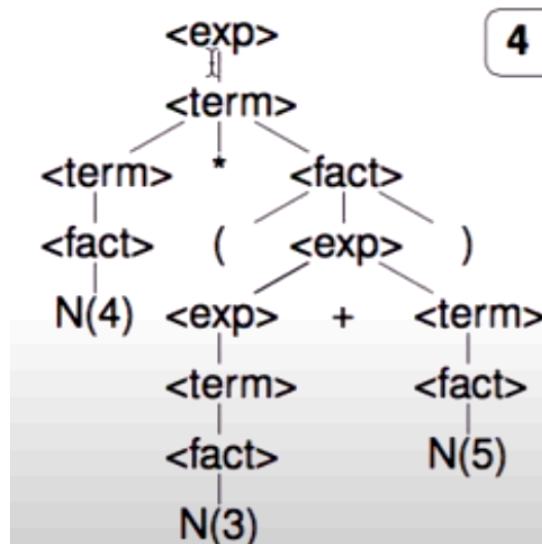
- **Bằng cách này, người ta sẽ giảm đi được số node của cây.**

### Ex2

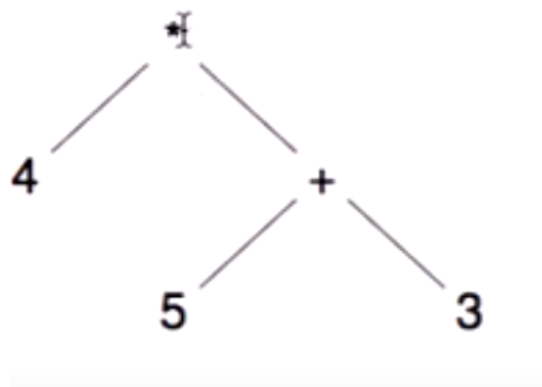
- Một ví dụ khác: Ta có văn phạm sau:

$\text{exp} \rightarrow \text{exp} + \text{term} \mid \text{term}$   
 $\text{term} \rightarrow \text{term} * \text{fact} \mid \text{fact}$   
 $\text{fact} \rightarrow ( \text{exp} ) \mid \text{ID} \mid \text{N}$

Từ đây, nếu cho một input là  $4 * (5 + 3)$ , ta sẽ sinh ra được một parse tree:



- Từ parse tree trên sinh ra cây cú pháp trừu tượng, ta sẽ sử dụng được cây cú pháp trừu tượng sau:



→ So với parse tree, AST Tree có những điểm vượt trội:

- Chỉ có 5 nodes so với 18 nodes của parse tree.
- chiều cao là 3 tầng so với 8 tầng



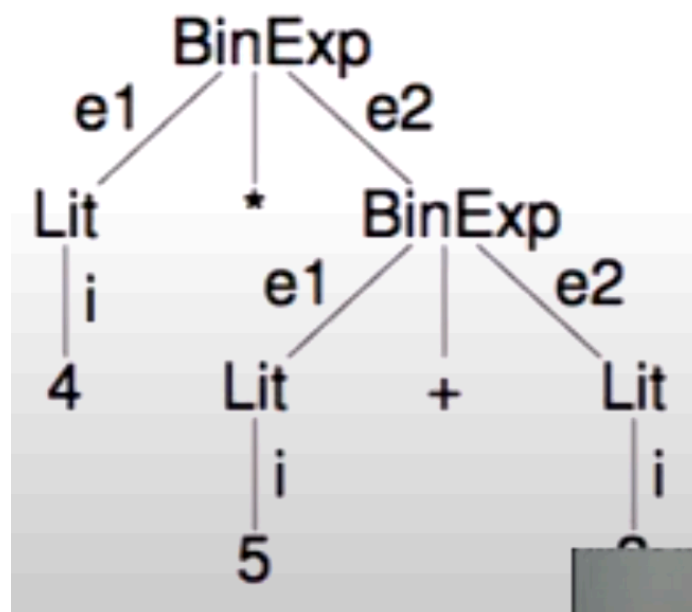
Kết luận: Cây cú pháp trừu tượng có thể giảm rất nhiều node so với parse tree, nhưng vẫn giữ được ngữ nghĩa.

## Hiện thực AST

- Từ lệnh:

```
trait Exp
case class BinExp(op:String,e1:Exp,e2:Exp) extends Exp
case class UnaExp(op:String,e:Exp) extends Exp
case class Lit(i:Int) extends Exp
```

- Với input là  $4 * (5 + 3)$ , ta được cây như sau:



Để viết biểu thức từ parse tree trên, ta cần khai báo theo quy tắc kết hợp và độ ưu tiên trong bài syntax analysis: `BinExp('*',Lit(4),BinExp('+',Lit(5), Lit(3)))`

# Cây phân tích cú pháp

- Các quy tắc được đưa ra để sinh code từ parse tree:

- Các non-terminal và terminal đều được định nghĩa như các node. Tuy nhiên, chúng có sự khác nhau về cách đặt tên:**
  - với các tín hiệu chưa kết thúc (non-terminal), tên của class khi định nghĩa sẽ được viết hoa chữ cái đầu + thêm cụm 'Context' phía sau.** Ví dụ: `stmt → StmtContext`, `expr → ExprContext`.
  - Với các tín hiệu kết thúc (terminal node), tên của class cũng chính là tên khi khai báo trong ANTLR4.** ví dụ: `ID → ID`, `ASSIGN → ASSIGN`.
  - Ví dụ lớn: Giả sử cho bộ rules sau:

```
grammar MC;
program: stmt;
stmt: assign | ifstmt;
assign: ID ASSIGN expr SEMI;
ifstmt: IF expr THEN stmt ELSE stmt;
expr: expr ADDOP term | term;
term: ID | INTLIT | LP expr LP;
```

- Khi đó, ta có class BAO GỒM CÁC NONTERMINAL được định nghĩa như sau:

```
class ProgramContext {
    class StmtContext;
    class AssignContext;
    class IfstmtContext;
    class ExprContext;
    class StmtContext;
    class TermContext;
}
```

- Các terminal node sẽ được định nghĩa trong cùng một class, tên là TerminalNode. Các non-terminal node sẽ được định nghĩa riêng cho từng tên, với rule đặt tên class ở mục 1.**

- Khác với cây nhị phân (có node con bên trái và phải), Với từng cây (từng input khác nhau), mỗi node cha tùy trường hợp sẽ có các node con khác nhau.**

- Ví dụ luật sinh sau: `expr: term ADDOP expr | term`
- Node non-terminal được sinh ra từ rule trên là ExprContext. Exprcontext có thể có 2 cách biểu diễn, tùy vào input:
  - ExprContext sinh ra 3 node con: TermContext, ADDOP, ExprContext.
  - ExprContext sinh ra node con duy nhất: TermContext.

3. Với mỗi văn phạm parse tree, mỗi symbol (kí tự terminal - non-terminal bất kì) trong một luật sẽ sinh ra một hoặc 2 phương thức.

a. Trường hợp 1 symbol sinh ra 1 phương thức: khi symbol đó là DUY NHẤT trong câu rule đó. lúc này, phương thức sẽ được gọi với datatype là class có nó (ví dụ như ExprContext là class của expr symbol).

Ví dụ: `assign: ID ASSIGN expr SEMI;` Với luật này, ta có thể xây dựng được một class `AssignContext` gồm 4 phương thức (vì các symbol `ID, ASSIGN, expr, SEMI` đều là duy nhất).

```
class AssignContext{
    TerminalNode ID(){...}
    TerminalNode ASSIGN(){...}
    ExprContext expr(){...}
    TerminalNode SEMI(){...}
}
```

b. Trường hợp 1 symbol sinh ra 2 phương thức: Khi các non-terminal bị trùng lại (một rule có từ 2 symbol loại đó trở lên), ta sinh ra đúng 2 luật cho tất cả các symbol giống nhau đó. Một luật trả về list các objectContext (không có tham số), luật còn lại trả về ObjectContext thứ i trong list định nghĩa ở trước. (luật này có 1 tham số là int i). Nếu ta gọi tham số i lớn hơn số phần tử, kết quả trả về là None.

Ví dụ: ta có luật sau: `ifstmt: IF expr THEN stmt ELSE stmt`. Ta có 2 stmt (non-terminal node) xuất hiện trong văn phạm trên. Như vậy, ta có thể sử dụng quy tắc 2 method:

```
class IfstmtContext{
    TerminalNode IF(){...} // IF - TERMINAL
    ExprContext expr(){...} // expr - unique non-terminal
    TerminalNode THEN(){...} // THEN - TERMINAL
    list<ExprContext> expr(){...} // first method - list
    ExprContext expr(int i){...} // second method - call to element i of list above
    TerminalNode ELSE(){...} // ELSE - TERMINAL
}
```

5. Trong xử lý văn phạm phi ngữ cảnh, ta rất hay gặp trường hợp có nhiều option trong cùng một rule. Để xử lý việc này, nên sử dụng phương thức `RuleContext getChild(int i)` để xử lý thống nhất các option trong cùng rule, thay vì rẽ nhánh.

RuleContext getChild(int i): *i*<sup>th</sup> child

```
stmt: assign      if (assign() != null)
                  //do something on assign()
      | ifstmt;    else
                  //do same thing on ifstmt()

                  //do something on getChild(0)
```

6. Để nhận dạng/phân loại các option, cơ bản nhất là so sánh số node của từng option. Phương thức `int getChildCount()` sẽ giúp ta xử lý theo độ dài của node. nếu `getChildCount() == i` với *i* là số node của một số option nào đó, ta sẽ phân nhánh và xử lý được.

int getChildCount(): number of children

```
term: ID          if (getChildCount() == 3)
                  //do something on exp()
      | INTLIT
      | LP exp RP; else
                  //do something on getChild(0)
```

## Visitor - ANTLR

- ANTLR sẽ sinh ra một phương thức visit cho mỗi ký hiệu không kết thúc (non-terminal node).

## Quiz

1. (Dr.Phung) Cho luật sinh của lệnh gán trong văn phạm của một ngôn ngữ như sau:

`assign → ID EQUAL exp SEMI`

trong đó ID là token đại diện cho một danh hiệu, EQUAL là token đại diện cho dấu gán '=', exp là ký hiệu không kết thúc đại diện cho biểu thức và SEMI là token đại diện cho dấu ';'. Ngữ nghĩa của lệnh gán là lấy giá trị của biểu thức exp gán vào biến có tên là ID. Giả sử trên cây AST, nút ASSIGN dùng để biểu diễn lệnh gán. Vậy nút này sẽ cần phải có bao nhiêu nút con?

- a. 4
- b. 3

c. 2

d. 1

→ Trả lời: Đầu tiên, xác định đề hỏi cần bao nhiêu NÚT CON của nút ASSIGN.

- Trong parse tree thì sẽ có 4 nút con: ID, EQUAL, exp, SEMI.
- Tuy nhiên, bản thân ASSIGN đã mang nghĩa "gán" → Không cần nút con EQUAL để biểu thị nữa. Có thể loại bỏ.
- Tiếp tục, Dấu SEMI không cần thiết cho lệnh gán, nó chỉ là biểu hiện của ngôn ngữ → loại
- Như vậy, nút ASSIGN còn 2 nút con: ID và exp.

2. (Dr.Phung) Cho văn phạm của lệnh For trên Pascal như sau:

`forstmt → FOR ID ASSIGN exp (TO | DOWNT0) exp DO stmt`

trong đó FOR, TO, DOWNT0, DO lần lượt là các token ứng với các từ khoá "for", "to", "downto" và "do"; ID là token ứng với biến chỉ số của lệnh for; ASSIGN là token ứng với ":="; exp là ký hiệu không kết thúc ứng với các biểu thức bắt đầu và kết thúc; stmt là ký hiệu không kết thúc ứng với thân vòng lặp for. Ngữ nghĩa của lệnh for trên Pascal là đầu tiên ID sẽ nhận trị của biểu thức thứ nhất, tùy theo lệnh for dùng TO hay DOWNT0 mà ID sẽ so sánh với trị của biểu thức thứ hai để thực hiện lặp stmt, sau mỗi lần lặp thì ID sẽ tăng hoặc giảm 1 (tùy theo TO hay DOWNT0).

Cho nút FORAST là nút ứng với lệnh for trên cây AST, vậy FORAST sẽ cần phải có bao nhiêu nút con để chứa đủ ngữ nghĩa của lệnh for.

a. 5

b. 8

c. 9

d. 4

→ Trả lời:

- Một node root FORAST có tối đa 8 nút: FOR, ID, ASSIGN, exp, (TO | DOWNT0), exp, DO, stmt. Điều này có nghĩa là 1 input. ta chỉ có thể dùng một trong 2 nút : TO hoặc DOWNT0. Vậy, loại câu C.
- Nút FOR đã được diễn giải luôn trong FORAST, nên cũng có thể bỏ mà không ảnh hưởng tới ngữ nghĩa của câu. Tương tự với ASSIGN và DO. Như vậy, loại FOR, ASSIGN, DO → Còn 5 nút.
- Tuy nhiên, không thể loại TO và DOWNT0 mà phải để option choice case này là 1 nút trong AST, vì nó ảnh hưởng trực tiếp tới ngữ nghĩa của cây: một cái đi lên, một cái đi xuống → Không thể loại nút TO/DOWNT0.
- Như vậy, từ node root FORAST, có 5 nút cần thiết giữ lại: ID, exp (1), (TO | DOWNT0), exp (2), stmt.

3. (Dr.Phung) Cho luật sinh của ký hiệu factor trong văn phạm của một biểu thức như sau:

`factor`  $\rightarrow$  `ID` | `INTLIT` | `FLOATLIT` | `LP exp RP`

trong đó ID là token mô tả một danh hiệu, INTLIT mô tả một hằng nguyên, FLOATLIT mô tả một hằng thực, LP và RP tương ứng là các token ứng với '(' và ')'; exp là ký hiệu văn phạm mô tả một biểu thức.

Cho biết các vế phải của luật sinh này sẽ tạo ra bao nhiêu nút con trên cây AST cho nút cha ứng với factor?

- a. 6
- b. 3
- c. 4
- d. 1

→ Trả lời:

- **Khi có nhiều trường hợp xảy ra (Hoặc này hoặc kia), ta lấy số node sinh ra của từng option, sau đó chọn số node TỐI ĐA trong các option.**
- Trong trường hợp này, node root factor sinh ra 4 branching: `ID` , `INTLIT` , `FLOATLIT` và `LP exp RP` . Xét từng nhánh:
  - Nhánh `ID` cần một node duy nhất chứa `ID`
  - Nhánh `INTLIT` cần một node duy nhất chứa `INTLIT`
  - Nhánh `FLOATLIT` cần một node duy nhất chứa `FLOATLIT` .
  - Nhánh `LP exp RP` cũng cần duy nhất 1 node chứa `exp` , `LP` và `RP` sẽ được lược bỏ mà không ảnh hưởng tới ngữ nghĩa.
- **Như vậy, 4 nhánh đều sinh tối đa 1 node, nên cũng chỉ cần 1 node con ở cây AST để biểu diễn cho văn phạm trên. Đáp án là 1 node.**

4. (DR.Phung) Chọn cây AST thích hợp nhất cho biểu thức:  $12 * (3 + 5) - 20$  (độ ưu tiên các phép toán như thông lệ)?

- a. `BinExp("*",Lit(12),BinExp("-",BinExp("+",Lit(3),Lit(5)),Lit(20)))`
- b. `BinExp("-",Lit(12),BinExp("*",BinExp("+",Lit(3),Lit(5)),Lit(20)))`
- c. `BinExp("-",Lit(20),BinExp("*",Lit(12),BinExp("+",Lit(3),Lit(5))))`
- d. **`BinExp("-",BinExp("*",Lit(12),BinExp("+",Lit(3),Lit(5))),Lit(20))`**

5. (Dr.Phung) Một giải pháp khác để biểu diễn biểu thức nhị phân là thay vì định nghĩa lớp BinOp như trong bài giảng, ta định nghĩa các lớp Plus, Minus, Mul and Div tương ứng cho các phép toán +, -, \* và /. Trong trường hợp này, một khai báo lớp Plus sẽ như thế nào?

- a. `case class Plus(op:String,e1:Exp,e2:Exp) extends Exp`

- b. `case class Plus(e1:Exp,e2:Exp)`
- c. **`case class Plus(e1:Exp,e2:Exp) extends Exp`**
- d. `case class Plus(e:Exp) extends Exp`

6. (Dr.Phung) Trong một ngôn ngữ lập trình tựa như Pascal có nhiều loại phát biểu khác nhau. Cho văn phạm của phát biểu gán trên ngôn ngữ này như sau:

`assign → ID ASSIGN exp SEMI`

trong đó ID đại diện cho một biến, ASSIGN đại diện cho dấu gán ":", exp đại diện cho biểu thức và SEMI đại diện cho dấu ";" để kết thúc phát biểu gán.

Cho Stmt là lớp trên cây AST đại diện cho bất kỳ phát biểu nào; kiểu ngữ nghĩa của ID là String, ASSIGN là String, exp là Expr và SEMI là String.

Hãy chọn định nghĩa thích hợp cho một phát biểu gán trên cây AST?

- a. `case class Assign(id:String,assign:String,exp:Expr,semi:String) extends Stmt\`
- b. `case class Assign(id:String,exp:Expr)`
- c. **`case class Assign(id:String,exp:Expr) extends Stmt`**
- d. `case class Assign(id:String) extends Stmt`

7. (Dr.Phung) Giả sử ta có luật sinh sau trong văn phạm:

`assign → ID ASSIGN exp SEMI`

Trên cây phân tích cú pháp (parse tree) sinh ra bởi ANTLR, nút ứng với luật này là đối tượng của lớp có tên là?

- a. assign
- b. Assign
- c. **AssignContext**

→ nonterminal là chữ thường, còn trong đối tượng sinh bởi node phải có tên là: **ObjectContext** (viết hoa chữ cái đầu + Đuôi "context")

8. (Dr. Phung) Cho luật sinh của một văn phạm như sau:

`assign → ID ASSIGN exp SEMI`

Giả sử biến ctx đang cất giữ nút ứng với assign trên cây phân tích cú pháp sinh ra bởi ANTLR. Hãy viết biểu thức để truy xuất nút ứng với exp bên vế phải?

- a. `ctx.ExpContext`
- b. `exp()`
- c. `ctx.getChild(2)`
- d. **`ctx.exp`**

**Note: c và d đều được, nhưng d chắc chắn hơn, bởi vì c nếu thay đổi thứ tự thì phải sửa code.**

9. (Dr. Phung) Cho văn phạm có các luật sinh sau:

$exp \rightarrow exp \text{ ADD } term \mid term$   
 $term \rightarrow term \text{ MUL } fact \mid fact$   
 $fact \rightarrow ID \mid INTLIT \mid LP \text{ exp } RP$

Với văn phạm trên, ANTLR sẽ sinh ra lớp Visitor có bao nhiêu phương thức visit chủ yếu?

- a. 7
- b. **3**
- c. 6
- d. 9

10. (Dr. Phung) Cho văn phạm của ngôn ngữ ABC có luật sinh sau:

$exp \rightarrow exp \text{ ADD } term \mid term$

Prototype của phương thức visit sinh ra bởi ANTLR cho luật sinh này là gì?

- a. T visitExpContext(ABCParser.Exp ctx)
- b. T visitExp(Exp ctx)
- c. **T visitExp(ABCParser.ExpContext ctx)**
- d. T visitExpContext(ABCParser.ExpContext ctx)

**Note:** Tên của visitor sẽ là 'visit' + tên của non-terminal với chữ cái đầu viết hoa.

11. Cho các luật sinh của văn phạm của ngôn ngữ ABC như sau:

$exp \rightarrow exp \text{ ADD } term \mid term$   
 $term \rightarrow term \text{ MUL } fact \mid fact$   
 $fact \rightarrow ID \mid INTLIT \mid LP \text{ exp } RP$

trong đó ADD là token đại diện phép toán +; MUL đại diện \*; ID cho danh hiệu; INTLIT cho số nguyên, LP cho { và RP cho }, và cấu trúc dữ liệu cho cây AST được định nghĩa (trên ngôn ngữ Scala) qua các lớp sau:

```
trait ExpAST
case class Plus(e1:ExpAST,e2:ExpAST) extends ExpAST
case class Mul(e1:ExpAST,e2:ExpAST) extends ExpAST
case class Ident(id:String) extends ExpAST
case class Intlit(val:Int) extends ExpAST
```

- a. Cho biết đối tượng Plus (lớp con của ExpAST) sẽ được tạo ra trong phương thức visit nào
  - a. Plus được tạo ra trong phương thức visitTerm
  - b. Plus được tạo ra trong phương thức visitExp
  - c. Plus được tạo ra trong phương thức visitFact
- b. Cho biết đối tượng Mul (lớp con của ExpAST) sẽ được tạo ra trong phương thức visit nào?

- a. Mul được tạo ra trong phương thức visitTerm
- b. Mul được tạo ra trong phương thức visitExp
- c. Mul được tạo ra trong phương thức visitFact
- c. **Cho biết đối tượng Id (lớp con của ExpAST) sẽ được tạo ra trong phương thức visit nào?**
  - a. Id được tạo ra trong phương thức visitExp
  - b. Id được tạo ra trong phương thức visitTerm
  - c. Id được tạo ra trong phương thức visitFact
  - d. Id được tạo ra trong phương thức visitID

## Lesson

- Số lượng con của một non-terminal node: `getChildCount()`
- Lấy ra chuỗi của một node: `getText()`